

CERTIFICAÇÃO LINUX

Privilege Escalation & Lateral Movement

Manual de Testes de QA — roteiros com exemplos práticos de requisição

Prof. Uirá Ribeiro

Material de apoio do curso Certificação Linux.

Todos os direitos reservados.

Manual de Testes de QA — Privilege Escalation e Lateral Movement

Referência: OWASP Top 10 2025 — A01 (Controle de Acesso Quebrado) e OWASP Testing Guide (WSTG). Público-alvo: QA / Testers funcionais e de segurança, sem necessidade de escrever exploits — foco em roteiros de teste reproduzíveis.

1. Escopo e definições

TERMO	DEFINIÇÃO	PERGUNTA QUE O TESTE RESPONDE
Privilege Escalation (Vertical)	Um usuário de baixo privilégio consegue executar ações reservadas a um perfil superior (ex.: usuário comum vira admin).	"Consigo fazer o que só o admin deveria fazer?"
Privilege Escalation (Horizontal)	Um usuário acessa dados/recursos de outro usuário do mesmo nível de privilégio.	"Consigo ver/editar dados de outra pessoa com o meu mesmo nível de acesso?"
Lateral Movement	Após comprometer/acessar um ponto do sistema (uma conta, um serviço, um host), o atacante se move para outros componentes, contas ou ambientes.	"A partir daqui, até onde mais eu consigo chegar?"

Pré-requisitos para o teste:

- Autorização formal (janela de teste, ambiente de homologação ou autorização explícita em produção).
- No mínimo 2–3 contas de teste por perfil (ex.: `user_a` , `user_b` , `admin` , `guest` , `support`).
- Acesso a um proxy de interceptação (Burp Suite Community/Pro, OWASP ZAP) e a um cliente HTTP (Postman, curl, httpie).
- Ambiente isolado — nunca testar escalonamento em produção sem sign-off explícito, dado o risco de efeitos colaterais (ex.: exclusão de dados reais, envio de notificações a usuários reais).

2. Metodologia geral (para os dois temas)

- 1. Mapear papéis e limites** — listar todos os perfis (roles), suas permissões documentadas e as fronteiras entre tenants/organizações/contas.
- 2. Mapear superfícies** — endpoints de API, telas administrativas, jobs em background, exportações, webhooks, integrações.
- 3. Testar com a menor credencial possível primeiro** — comece sempre pelo perfil de menor privilégio (guest → user → poder → admin), nunca pelo mais alto.
- 4. Comparar resposta esperada vs. observada** — toda tentativa deve ter uma expectativa clara (ex.: "deveria retornar 403") registrada *antes* de executar o teste.
- 5. Testar em ambos os sentidos:** UI (front-end esconde botão, mas back-end permite?) e API direta (chamando o endpoint sem passar pela UI).
- 6. Registrar evidências:** request/response completos (headers, body), timestamp, conta usada, resultado esperado x obtido.

3. Testes de Privilege Escalation

3.1 Escalonamento Vertical

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO	FERRAMENTA
PE-V-01	Acesso direto a endpoint administrativo sem permissão	Logar como usuário comum. Requisitar diretamente a URL/endpoint de admin (ex.: <code>/admin/users</code> , <code>/api/v1/admin/settings</code>) via navegador e via chamada de API direta.	HTTP 403/401 em ambos os casos, mesmo que a UI não mostre o link.	Burp/ZAP, curl
PE-V-02	Manipulação de parâmetro de role/perfil	Interceptar requisição de cadastro/edição de perfil e alterar campos como <code>role</code> , <code>isAdmin</code> , <code>permission_level</code> , <code>accountType</code> para valor de privilégio maior. Reenviar.	Servidor rejeita ou ignora o campo alterado; role permanece a original.	Burp Repeater
PE-V-03	Elevação via IDOR em endpoint de atualização	Enviar <code>PUT/PATCH /users/{id}</code> alterando o próprio <code>id</code> para o de um admin, tentando também setar <code>role=admin</code> no payload.	403/404, sem alteração de dados de terceiros.	Postman
PE-V-04	Reuso de token/JWT com claims adulterados	Decodificar JWT (jwt.io), alterar claim de role/scope, testar se a assinatura é validada. Testar também <code>alg=none</code> .	Token adulterado é rejeitado (assinatura inválida / 401).	jwt.io, Burp

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO	FERRAMENTA
PE-V-05	Funções administrativas expostas em endpoints "ocultos"	Buscar em JS do front-end, mapa de rotas, Swagger/OpenAPI exposto, por rotas não referenciadas na UI (/api/internal/* , /api/debug/*). Testar acesso com conta comum.	Rotas internas exigem autenticação/autorização adequada ou não estão acessíveis externamente.	Burp Spider, gau, ffuf
PE-V-06	Escalonamento via criação de recurso com permissões elevadas	Como usuário comum, tentar criar um recurso (ex.: chave de API, webhook, usuário) definindo permissões/escopo administrativo no corpo da requisição.	Sistema ignora/rejeita o campo de escopo elevado.	Postman
PE-V-07	Bypass de verificação client-side	Identificar checagens de role feitas apenas em JavaScript (botão desabilitado/escondido). Chamar a ação diretamente via API.	Back-end reforça a mesma regra independente do front-end.	DevTools + curl
PE-V-08	Escalonamento por fluxo de "impersonation"/suporte	Se existir função de "logar como outro usuário" para suporte, testar se um usuário comum consegue acioná-la ou se um agente de suporte consegue impersonar um admin sem trilha de auditoria.	Função restrita a perfis corretos e com log de auditoria.	Manual
PE-V-09	Escalonamento via reset de senha / fluxo de convite	Testar se o link de convite/reset de senha de admin pode ser reaproveitado para criar conta com privilégio superior ao pretendido.	Convite associado estritamente ao role definido na emissão.	Manual
PE-V-10	Cache/CDN vazando resposta de admin para usuário comum	Requisitar página administrativa como admin, depois a mesma URL como usuário comum, checando se resposta em cache (CDN/proxy) é servida.	Nenhuma resposta sensível de outro usuário é cacheada/servida.	curl -I, headers Cache-Control

Exemplos práticos — Escalonamento Vertical

PE-V-01 — Acesso direto a endpoint administrativo

```
# Logado como usuário comum (token capturado do navegador/DevTools)
curl -i https://api.exemplo.com/admin/users \
  -H "Authorization: Bearer <token_user_comum>"
```

Esperado: HTTP/1.1 403 Forbidden . Se vier 200 OK com a lista de usuários, é falha crítica — repita também pela URL direta no navegador (<https://app.exemplo.com/admin/users>), já que a UI pode simplesmente esconder o link.

PE-V-02 — Manipulação de parâmetro de role Capture no proxy a requisição de edição do próprio perfil:

```
PATCH /api/users/42
{"name": "Maria Silva", "email": "maria@exemplo.com"}
```

Altere o corpo no Burp Repeater adicionando os campos de privilégio e reenvie:

```
{"name": "Maria Silva", "email": "maria@exemplo.com", "role": "admin", "isAdmin": true}
```

Esperado: os campos `role` / `isAdmin` são ignorados pelo servidor. Confirme com um `GET /api/users/42` logo em seguida — o role deve continuar o original.

PE-V-03 — IDOR em update setando role

```
PUT /api/users/17 HTTP/1.1
Host: api.exemplo.com
Authorization: Bearer <token_do_user_17>
Content-Type: application/json

{"id": 3, "role": "admin"}
```

onde 3 é o ID de uma conta admin. Esperado: 403 ou 404 , e nenhuma alteração no usuário 3.

PE-V-04 — JWT com claims adulterados

1. Copie o JWT do header `Authorization` .
2. Decodifique em jwt.io — payload típico: `{"sub":"1042","role":"user","exp":1893456000}` .
3. Troque `"role":"user"` por `"role":"admin"` e reenvie sem recalculer a assinatura (reproveite a assinatura antiga) e depois teste também `"alg":"none"` :

```
curl -i https://api.exemplo.com/me \
-H "Authorization: Bearer eyJhbGciOiJIub25lIn0.eyJzdWIiOiIxMDQyIiwicm9sZSI6ImFkbWluIn0."
```

Esperado: 401 Unauthorized em ambos os casos — assinatura inválida e `alg=none` devem ser rejeitados.

PE-V-05 — Descoberta de rotas administrativas ocultas

```
# via wordlist de endpoints comuns de API
ffuf -u https://api.exemplo.com/api/FUZZ -w wordlist-api-endpoints.txt -mc 200,301,302,403

# ou minerando o bundle JS do front-end por rotas não linkadas na UI
curl -s https://app.exemplo.com/static/js/main.js | grep -oE '"/api/[a-zA-Z0-9_/-]+"' | sort
-u
```

Para cada rota encontrada, repita o teste PE-V-01 com uma conta comum.

PE-V-06 — Criação de recurso com escopo elevado

```
POST /api/api-keys HTTP/1.1
Authorization: Bearer <token_user_comum>
Content-Type: application/json

{"name": "minha-chave", "scope": "admin:*"}
```

Esperado: o escopo retornado/gravado é o máximo permitido ao usuário (ex.: `read:own`), nunca `admin:*`.

PE-V-07 — Bypass de verificação client-side No DevTools, o botão "Excluir usuário" está disabled para o perfil comum. Chame a ação direto na API, ignorando a UI:

```
curl -X DELETE https://api.exemplo.com/users/99 -H "Authorization: Bearer <token_user_comum>"
```

Esperado: `403`, independentemente do botão estar oculto/desabilitado na tela.

PE-V-08 — Impersonation sem trilha de auditoria

```
POST /support/impersonate HTTP/1.1
Authorization: Bearer <token_user_comum_ou_agente_suporte>
Content-Type: application/json

{"userId": 501}
```

Teste com conta comum (deve falhar) e, depois, com conta de suporte legítima (deve funcionar, mas gerar registro em `/audit-log` com quem impersonou, quando e por quê).

PE-V-09 — Reset de senha/convite reaproveitado

1. Gere um convite de admin: `POST /invites {"email":"novo.admin@exemplo.com","role":"admin"}`.
2. Intercepte o link/token do convite antes do envio por e-mail.
3. Aceite o convite alterando o e-mail/senha para dados que você controla:

```
POST /invites/accept HTTP/1.1
Content-Type: application/json

{"token": "<token_do_convite>", "email": "atacante@exemplo.com", "password": "Senha123!"}
```

Esperado: o role da conta final ainda deve ser estritamente o definido na emissão do convite — teste se não é possível também sobrescrever o `role` neste payload.

PE-V-10 — Vazamento de página admin via cache/CDN

```
curl -sI https://app.exemplo.com/dashboard/admin-report -H "Authorization: Bearer
<admin_token>"
# observe headers Cache-Control / Age / X-Cache

curl -sI https://app.exemplo.com/dashboard/admin-report -H "Authorization: Bearer
<user_token>"
# se retornar 200 com X-Cache: HIT em vez de 403, há vazamento via cache
```

3.2 Escalonamento Horizontal

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
PE-H-01	IDOR clássico (Insecure Direct Object Reference)	Como <code>user_a</code> , capturar URL/endpoint que referencia um ID de recurso próprio (ex.: <code>/invoices/1001</code>). Trocar o ID pelo de <code>user_b</code> (<code>/invoices/1002</code>).	403/404 — não retorna dado de outro usuário.
PE-H-02	IDOR em parâmetros não visíveis na URL	Repetir PE-H-01 usando IDs em body JSON, headers customizados ou parâmetros de query ocultos (ex.: <code>account_id</code> num POST).	Mesma proteção aplicada independentemente de onde o ID trafega.
PE-H-03	Enumeração sequencial de IDs	Se IDs são sequenciais/numéricos, iterar uma faixa (script simples ou Intruder) para verificar quantos registros de outros usuários são acessíveis.	Nenhum registro de terceiros retornado; idealmente IDs não previsíveis (UUID).
PE-H-04	Autorização em operações de escrita, não só leitura	Repetir os testes acima para <code>PUT</code> , <code>PATCH</code> , <code>DELETE</code> , não só <code>GET</code> . Muitas apps validam owner só na leitura.	Toda operação de escrita valida o dono do recurso.
PE-H-05	Multi-tenancy / cross-tenant	Em SaaS multi-tenant, testar se um usuário da Empresa A acessa dados/relatórios/configurações da Empresa B trocando <code>tenant_id</code> / <code>org_id</code> / subdomínio.	Isolamento total entre tenants.

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
PE-H-06	Exportação/relatórios em lote	Testar endpoints de exportação (CSV/PDF/Excel) e de busca/filtros — frequentemente esquecem a filtragem por owner que existe na tela principal.	Exportação respeita o mesmo escopo de autorização da tela.
PE-H-07	Funcionalidades de compartilhamento	Testar se um link de "compartilhar" gerado por <code>user_a</code> concede acesso além do pretendido, ou se pode ser adivinhado/reaproveitado por <code>user_b</code> .	Compartilhamento com escopo e expiração corretos.
PE-H-08	Autorização em WebSockets / eventos em tempo real	Conectar via WebSocket como <code>user_a</code> e verificar se eventos/mensagens destinados a <code>user_b</code> vazam no mesmo canal.	Mensagens segregadas por sessão/usuário autorizado.
PE-H-09	Mass assignment em atualização de perfil	Enviar campos extras não esperados no PATCH do próprio perfil (ex.: <code>owner_id</code> , <code>linked_account_id</code>) apontando para outro usuário.	Campos não previstos são ignorados pelo servidor.

Exemplos práticos — Escalonamento Horizontal

PE-H-01 — IDOR clássico

```
curl -i https://api.exemplo.com/invoices/1001 -H "Authorization: Bearer <token_user_a>"
curl -i https://api.exemplo.com/invoices/1002 -H "Authorization: Bearer <token_user_a>"
```

Se `1002` pertence a `user_b` e retorna `200` com os dados da fatura, IDOR confirmado.

PE-H-02 — IDOR oculto em body/headers

```
POST /api/reports/generate HTTP/1.1
Authorization: Bearer <token_user_a>
Content-Type: application/json

{"accountId": 2002, "period": "2026-08"}
```

onde `2002` é a conta de `user_b`. Repita trocando o ID em headers customizados (`X-Account-Id: 2002`) — muitas implementações só validam o ID quando ele está na URL.

PE-H-03 — Enumeração sequencial de IDs No Burp Intruder, configure o path `/invoices/$1000$`, payload numérico `1000 – 1100`, e compare tamanho/status de cada resposta para identificar quantos registros de terceiros retornam `200`.

PE-H-04 — Autorização em operações de escrita


```
curl -X DELETE https://api.exemplo.com/documents/455 -H "Authorization: Bearer <token_user_a>"
```

onde 455 pertence a `user_b`. Esperado: 403 — muitas apps só checam o dono na leitura (GET), não na escrita.

PE-H-05 — Cross-tenant

```
GET /api/reports?tenant_id=empresa-b HTTP/1.1
Authorization: Bearer <token_empresa_a>
```

ou trocar o subdomínio mantendo o mesmo cookie de sessão: `empresa-a.app.exemplo.com` → `empresa-b.app.exemplo.com`.

PE-H-06 — Exportação sem filtro de owner

```
curl "https://api.exemplo.com/export/csv?from=2026-01-01&to=2026-08-01" \
-H "Authorization: Bearer <token_user_a>" -o export.csv
```

Abra `export.csv` e verifique se há linhas pertencentes a outros usuários — a tela principal filtra por owner, mas a exportação em lote frequentemente esquece esse filtro.

PE-H-07 — Link de compartilhamento previsível

```
curl -i "https://app.exemplo.com/shared/doc?token=AbC123"
```

Verifique se o `token` é curto/sequencial (teste variações com ffuf) e se expira após uso/tempo.

PE-H-08 — Vazamento em WebSocket Conecte como `user_a` (`wss://api.exemplo.com/live?token=<token_user_a>`) e, no console do navegador, registre as mensagens recebidas — verifique se aparecem eventos com `userId` de outros usuários no mesmo canal.

PE-H-09 — Mass assignment no próprio perfil

```
PATCH /api/profile HTTP/1.1
Authorization: Bearer <token_user_a>
Content-Type: application/json

{"displayName": "Ana", "linked_account_id": 77}
```

onde 77 é a conta de outro usuário. Esperado: o campo `linked_account_id` é ignorado ou validado contra ownership.

Checklist rápido de Privilege Escalation:

- ☐ Toda ação administrativa é revalidada no back-end (não só escondida na UI)?
- ☐ IDs de recursos não são previsíveis, e mesmo que sejam, há checagem de ownership?
- ☐ Tokens/JWT têm assinatura validada e claims de role não são confiáveis vindas do cliente?
- ☐ Operações de escrita (não só leitura) são protegidas?
- ☐ Exportações e buscas em lote respeitam o mesmo escopo de autorização das telas?
- ☐ Existe log de auditoria para ações sensíveis (mudança de role, impersonation)?

4. Testes de Lateral Movement

No contexto de aplicação web (não infraestrutura de rede pura), lateral movement é testado a partir de um ponto de acesso já obtido (uma conta, uma sessão, um serviço) para verificar até onde esse acesso se propaga.

4.1 Movimentação entre contas/sessões

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
LM-01	Session fixation / reuso de sessão	Fixar um <code>session_id</code> antes do login de outro usuário e verificar se a sessão passa a ser válida para a conta autenticada.	Sessão é regenerada no login; ID antigo invalidado.
LM-02	Roubo de token entre subdomínios/serviços	Se a app usa múltiplos subdomínios (<code>app.</code> , <code>admin.</code> , <code>api.</code>), verificar se um cookie/token de um domínio é aceito em outro sem revalidação de escopo.	Cookies com <code>Domain</code> / <code>SameSite</code> restritos; tokens com <code>audience</code> (<code>aud</code>) validado por serviço.
LM-03	SSO / propagação indevida de privilégio entre sistemas	Autenticar via SSO em um sistema de baixo privilégio e testar se o mesmo token dá acesso a um sistema integrado de maior criticidade (ex.: portal de RH → sistema financeiro).	Escopos (<code>scope</code> / <code>aud</code>) segregados por sistema; token de um serviço não é aceito por outro sem novo consentimento.

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
LM-04	Pivot via API keys/tokens de integração	Se a conta comprometida tem uma integração ativa (ex.: chave de API para um serviço terceiro), testar se essa chave permite acessar dados de outros usuários/tenants no serviço integrado.	Chave de API escopada estritamente ao tenant/usuário emissor.
LM-05	Movimentação via funcionalidades de "convite"/delegação	Testar se um usuário comprometido pode se auto-convitar/promover para outros times, grupos ou workspaces que não deveria acessar.	Convite/adesão a grupo exige aprovação do dono/admin do grupo alvo.
LM-06	Reuso de credenciais entre ambientes	Testar (com autorização) se credenciais de homologação funcionam em produção ou vice-versa, indicando falta de segregação de ambientes.	Credenciais e segredos segregados por ambiente.

Exemplos práticos — Contas e Sessões

LM-01 — Session fixation

```
# Passo 1: obter um session id sem estar logado
curl -i https://app.exemplo.com/login -c cookies.txt

# Passo 2: efetuar login normalmente reaproveitando o mesmo cookie (-b e -c no mesmo arquivo)
curl -i https://app.exemplo.com/login -b cookies.txt -c cookies.txt \
  -d "user=vitima@exemplo.com&pass=SenhaVitima123"

# Passo 3: checar se o Set-Cookie mudou o valor da sessão após o login
curl -i https://app.exemplo.com/me -b cookies.txt
```

Esperado: o `Set-Cookie` do passo 2 traz um `session_id` novo — se o valor do passo 1 continuar válido e autenticado, é session fixation.

LM-02 — Cookie aceito entre subdomínios

```
curl -i https://admin.exemplo.com/internal/ping -H "Cookie:
session=<cookie_capturado_em_app.exemplo.com>"
```

Verifique o atributo `Domain=.exemplo.com` no `Set-Cookie` original — se for amplo demais, o mesmo cookie pode autenticar em `admin.` ou `api.`.

LM-03 — Token de SSO aceito em sistema de maior criticidade

```
curl -i https://financeiro.exemplo.com/api/me \
-H "Authorization: Bearer <token_emitido_para_aud=portal-rh>"
```

Esperado: 401 — o aud (audience) do token não corresponde ao serviço financeiro; decodifique o JWT em jwt.io para confirmar o valor de aud / scope antes do teste.

LM-04 — Pivot via API key de integração

```
curl -i "https://api.exemplo.com/v1/customers?tenant=outro-tenant-id" \
-H "X-API-Key: <chave_criada_para_integracao_zapier_do_tenant_a>"
```

Esperado: 403 — a chave deve ser escopada ao tenant que a emitiu, mesmo trocando o parâmetro de tenant na URL.

LM-05 — Auto-adesão a time/grupo

```
POST /api/teams/42/members HTTP/1.1
Authorization: Bearer <token_user_comprometido>
Content-Type: application/json

{"userId": "self"}
```

Esperado: 403 sem convite/aprovação prévia do dono do time 42 .

LM-06 — Credenciais reaproveitadas entre ambientes

```
# Somente com autorização explícita do responsável pelo ambiente de produção
curl -i https://api.producao.exemplo.com/login -d '{"user":"qa_homolog","pass":"SenhaHomolog123"}'
```

Esperado: falha de autenticação — credenciais de homologação não devem existir/funcionar em produção.

4.2 Movimentação entre serviços/componentes internos

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
LM-07	SSRF como vetor de pivot	Identificar campos que fazem requisições server-side (upload por URL, webhooks, preview de link) e testar acesso a endpoints internos (<code>http://169.254.169.254</code> , <code>http://localhost:PORT</code> , serviços internos por IP privado).	Requisições a ranges internos/metadata bloqueadas (allowlist de destinos).

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
LM-08	Acesso a serviços internos via API Gateway mal segmentado	Testar se, autenticado em um microserviço de baixo risco, é possível chamar diretamente (via gateway) endpoints de microserviços internos que deveriam ser privados (service-to-service only).	Endpoints internos não expostos publicamente / exigem mTLS ou token de serviço.
LM-09	Escalonamento via filas/jobs assíncronos	Verificar se é possível injetar/alterar payload de uma mensagem em fila (se exposta) para que o worker execute ação com privilégio do worker (que normalmente é alto) em nome do atacante.	Payloads de fila validados e sem confiar em dados de origem não confiável.
LM-10	Credenciais expostas em variáveis de ambiente/logs acessíveis	Testar se endpoints de debug, stack traces, ou arquivos <code>.env</code> / <code>.git</code> expostos revelam credenciais que permitem acesso a outro serviço (DB, storage, serviço terceiro).	Nenhuma credencial exposta; debug desabilitado em produção.
LM-11	Movimentação via storage compartilhado (buckets/containers)	Testar se um usuário/serviço com acesso a um bucket/pasta consegue navegar (path traversal, listagem) para áreas de outros tenants/serviços no mesmo storage.	Isolamento por prefixo/bucket com políticas de acesso (IAM) restritivas.
LM-12	Reuso de conexão de banco compartilhada	Se múltiplos serviços compartilham a mesma instância de banco, testar (via SQLi ou acesso a endpoints de relatório) se é possível consultar tabelas/schemas de outro serviço/tenant.	Segregação por schema/usuário de banco com permissões mínimas (least privilege).

Exemplos práticos — Serviços e Componentes Internos

LM-07 — SSRF como vetor de pivot

```
POST /api/webhooks/test HTTP/1.1
Authorization: Bearer <token>
Content-Type: application/json

{"url": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"}
```

Teste também `http://localhost:8080/`, `http://127.0.0.1:6379/` (Redis) e IPs de rede interna (`10.0.0.0/8`, `192.168.0.0/16`). Esperado: a requisição server-side é bloqueada/rejeitada para ranges internos e metadata de nuvem.

LM-08 — Gateway expondo serviço interno

```
curl -i https://gateway.exemplo.com/internal/billing-service/adjust-balance \
-H "Authorization: Bearer <token_de_servico_de_baixo_risco>"
```

Esperado: 403 / 404 — rotas internas (service-to-service) não devem ser roteáveis por um token de serviço externo comum.

LM-09 — Payload malicioso injetado em fila Se houver um painel de fila acessível em homologação (RabbitMQ Management, console SQS), publique manualmente:

```
{"jobType": "sendInvoice", "userId": 10, "amount": -999999}
```

Esperado: o worker valida o payload (tipo, faixa de valores, permissão do `userId`) antes de processar, em vez de confiar cegamente na mensagem.

LM-10 — Credenciais expostas em debug/arquivos

```
curl -s -o /dev/null -w "%{http_code}\n" https://app.exemplo.com/.env
curl -s -o /dev/null -w "%{http_code}\n" https://app.exemplo.com/debug/vars
curl -s -o /dev/null -w "%{http_code}\n" https://app.exemplo.com/.git/config
```

Esperado: 404 nos três — nenhum deles deve estar acessível publicamente.

LM-11 — Path traversal em storage compartilhado

```
curl -i "https://storage.exemplo.com/tenants/empresa-a/../../empresa-b/relatorio.pdf" \
-H "Authorization: Bearer <token_empresa_a>"

# ou, testando políticas de bucket diretamente
aws s3 ls s3://exemplo-uploads/tenants/ --profile empresa-a-creds
```

Esperado: 403 no traversal e listagem restrita apenas ao prefixo do próprio tenant.

LM-12 — Consulta cross-schema em banco compartilhado

```
GET /api/reports/raw?query=SELECT%20*%20FROM%20outro_servico.usuarios HTTP/1.1
Authorization: Bearer <token_servico_a>
```

Esperado: erro de permissão do banco — o usuário de banco do serviço A não deve enxergar o schema do serviço B.

4.3 Movimentação entre perfis dentro da mesma aplicação

#	CASO DE TESTE	PASSOS	RESULTADO ESPERADO
LM-13	De "read-only" para "write" em outro módulo	Usuário com acesso de leitura a um módulo tenta usar o mesmo token/sessão para escrever em outro módulo relacionado.	Autorização por módulo/ação, não apenas por autenticação global.
LM-14	De conta de teste/demo para conta real	Testar se contas de demonstração/sandbox conseguem, por reuso de endpoint ou parâmetro, alcançar dados de contas de produção.	Ambientes de demo totalmente isolados de dados reais.
LM-15	Encadeamento de vulnerabilidades (chaining)	Combinar um IDOR (item 3.2) para obter um ID de outro usuário, e então usar esse ID em um segundo endpoint (ex.: reset de senha, exportação, API key) para expandir o acesso.	Cada endpoint reforça sua própria checagem de autorização, quebrando a cadeia.

Exemplos práticos — Perfis e Módulos

LM-13 — De read-only para write em outro módulo

```
curl -X POST https://api.exemplo.com/analytics/config \
-H "Authorization: Bearer <token_role_viewer>" \
-H "Content-Type: application/json" \
-d '{"retentionDays": 1}'
```

Esperado: 403 — um token com escopo `analytics:read` não deve permitir `POST` de configuração em nenhum módulo, nem correlatos.

LM-14 — De conta demo para dados de produção

```
curl -i https://api.exemplo.com/v1/customers/1 -H "Authorization: Bearer <token_conta_demo>"
```

Esperado: 403 / 404 — o cliente de ID `1` (real, de produção) não deve ser alcançável a partir de um token/ambiente de demonstração.

LM-15 — Chaining: IDOR → reset de senha → account takeover

```
# Passo 1 – IDOR (ver PE-H-01) revela o ID de outro usuário
curl -i https://api.exemplo.com/users/2044 -H "Authorization: Bearer <token_user_a>"

# Passo 2 – usa esse ID no fluxo de reset de senha
curl -i https://api.exemplo.com/password-reset \
  -H "Content-Type: application/json" \
  -d '{"userId": 2044}'
```

Esperado: o endpoint de reset não deve expor o link/token de reset na resposta da API (só deve ser enviado por e-mail ao dono da conta). Se a resposta do passo 2 retornar o link diretamente, a cadeia PE-H-01 + LM-15 permite takeover completo da conta 2044 sem qualquer credencial dela.

Checklist rápido de Lateral Movement:

- ☐ Sessões são regeneradas em login/mudança de privilégio (evita session fixation)?
- ☐ Cookies/tokens são escopados por domínio/serviço/tenant (`aud` , `SameSite` , `Domain`)?
- ☐ Chamadas server-side a URLs arbitrárias (SSRF) são bloqueadas para ranges internos?
- ☐ Microserviços internos não são alcançáveis diretamente por clientes externos?
- ☐ Segredos/credenciais não vazam em logs, debug, `.env` , repositórios?
- ☐ Storage e banco de dados compartilhados têm isolamento lógico por tenant/serviço?
- ☐ Testou-se o encadeamento de duas ou mais falhas menores (chaining), não só cada uma isolada?

5. Roteiro sugerido de execução (sprint de teste)

1. **Preparação** (0.5 dia): criar contas de teste para cada perfil/tenant, mapear endpoints (Swagger, proxy passivo navegando o app), definir matriz de permissões esperada.
2. **Execução Privilege Escalation** (1–2 dias): rodar 3.1 e 3.2 sistematicamente, endpoint por endpoint, usando a matriz como oráculo (o que deveria/não deveria funcionar).
3. **Execução Lateral Movement** (1–2 dias): a partir de cada perfil comprometido "hipoteticamente", tentar os cenários da seção 4.
4. **Chaining** (0.5 dia): tentar combinar achados (ex.: IDOR + SSRF, mass assignment + impersonation).
5. **Relato**: para cada achado, registrar:
 - Perfil/conta usada e perfil alvo.
 - Endpoint/tela, request/response completos.

- Severidade (referenciar CVSS ou critério interno) e impacto de negócio.
- Evidência (screenshot, request salvo no Burp/Postman collection).
- Sugestão de correção (ex.: validar ownership no back-end, aplicar RBAC por endpoint).

6. Ferramentas de apoio

FERRAMENTA	USO
Burp Suite / OWASP ZAP	Interceptação, repetição e automação de requisições (Repeater/Intruder para IDOR e enumeração).
Postman/Insomnia	Criar coleções com variáveis de perfil (<code>user_a_token</code> , <code>admin_token</code>) para alternar contexto rapidamente.
jwt.io / jwt_tool	Inspecionar e adulterar JWTs para testar validação de assinatura/claims.
ffuf / gobuster	Descobrir rotas/endpoints ocultos ou não documentados.
curl / httpie	Testes rápidos e reproduzíveis em CI, fora da UI.
Autorize (extensão Burp)	Automatiza a comparação de resposta entre um usuário de baixo privilégio e um de alto privilégio para o mesmo endpoint.

7. Referências

- OWASP Top 10 2025 — A01: Broken Access Control.
- OWASP Web Security Testing Guide (WSTG) — seção "Testing for Authorization" e "Testing for Session Management".
- OWASP API Security Top 10 — API1 (BOLA/IDOR) e API5 (Broken Function Level Authorization).
- MITRE ATT&CK — táticas TA0004 (Privilege Escalation) e TA0008 (Lateral Movement), como referência conceitual (mapeamento para infraestrutura, adaptável ao contexto de aplicação web acima).